

Regresión Logística y modelos de clasificación con Scikit-Learn

Informe de implementación de algoritmos de Machine Learning Supervisado
CADI Introducción A Machine Learning · Semana 8

Estudiante: Ruby Dayana Cárdenas Gómez

Institución: Universidad de Cundinamarca

Programa: CADI Introducción A Machine Learning · – Especialización en Inteligencia Artificial

Fecha: 8 de Octubre de 2026

Repositorio en GitHub: <https://github.com/RubyDayana/ml-supervisado-clasificacion.git>

1. Introducción

Después de estudiar en qué consiste el aprendizaje supervisado y cuáles son sus algoritmos principales, en esta semana los implementé de verdad. El objetivo fue construir, en Google Colab y con Scikit-Learn, un ejercicio de Regresión Logística y los cinco algoritmos de clasificación vistos en la guía (Árbol de Decisión, Random Forest, SVM, KNN y Naive Bayes), entrenarlos sobre el mismo conjunto de datos y compararlos con las métricas de precisión, recall, F1 y matriz de confusión.

Elegí el dataset de cáncer de mama de Wisconsin, que viene incluido en Scikit-Learn (Pedregosa et al., 2011): 569 pacientes, 30 características numéricas medidas sobre un tumor (radio, textura, perímetro, área, etc.) y una etiqueta que indica si el tumor es **maligno (0)** o **benigno (1)**. Lo escogí por tres razones: ya viene etiquetado, es un problema de clasificación binaria limpio para empezar, y es un caso real donde las métricas tienen un significado concreto: un falso negativo es decirle a una paciente que su tumor es benigno cuando no lo es.

El cuaderno completo, con el código y todas las salidas, está en el repositorio de GitHub: <https://github.com/RubyDayana/ml-supervisado-clasificacion.git>. Las capturas de este informe son de ese cuaderno de nombre `ml_supervisado_rdcardenas.ipynb`.

2. Fundamentos

2.1 Aprendizaje supervisado y clasificación

En el aprendizaje supervisado el modelo recibe ejemplos con la respuesta correcta y aprende la relación entre las entradas y la salida (Gupta et al., 2022). Cuando la salida es una etiqueta, como aquí (maligno o benigno), el problema es de clasificación. Para evaluar honestamente se separa una parte de los datos que el modelo nunca ve durante el entrenamiento (Géron, 2019); en este ejercicio usé 70 % para entrenar (398 pacientes) y 30 % para probar (171 pacientes).

2.2 Regresión logística

Aunque se llame regresión, es un clasificador. Calcula una combinación lineal de las entradas, $z = w \cdot x + b$, y la pasa por la función sigmoide, que convierte cualquier número en una probabilidad entre 0 y 1. Si la probabilidad es mayor que 0,5 el modelo predice la clase 1 (Géron, 2019). Los pesos w se aprenden minimizando el error sobre los datos de entrenamiento.

2.3 Las métricas

La **matriz de confusión** cruza lo que el modelo predijo con lo que era verdad y muestra cuatro cantidades: verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos. A partir de ella se calculan la **exactitud** (aciertos totales), la **precisión** (de los que el modelo dijo que eran benignos, cuántos lo eran de verdad), el **recall** (de los que eran benignos, cuántos encontró el modelo) y el **F1**, que es el promedio armónico de las dos anteriores (Géron, 2019).

3. Desarrollo en Google Colab

3.1 Librerías y datos

Cargué Pandas, NumPy, Matplotlib, Seaborn y los módulos de Scikit-Learn necesarios (versión 1.8.0). El dataset tiene 569 filas y 30 columnas; 357 pacientes son benignos y 212 malignos, así que está ligeramente desbalanceado. Por eso usé stratify al dividir, para que la proporción se mantenga en ambos conjuntos.

```
O. Librerías

import numpy as np # números y matrices
import pandas as pd # tablas de datos
import matplotlib.pyplot as plt # gráficas
import seaborn as sns # matriz de confusión

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report, ConfusionMatrixDisplay)

import sklearn
print("Scikit-Learn:", sklearn.__version__)
print("Pandas:", pd.__version__)
print("NumPy:", np.__version__)

Scikit-Learn: 1.6.1
Pandas: 2.2.3
NumPy: 2.1.3
```

Captura 1. Librerías y versiones.

```
1. Cargar y explorar los datos

El dataset ya viene etiquetado, así que es un problema de aprendizaje supervisado de clasificación binaria.

datos = load_breast_cancer()
X = pd.DataFrame(datos.data, columns=datos.feature_names) # las 30 características (entradas)
y = pd.Series(datos.target, name="diagnostico") # la etiqueta (salida): 0 = maligno, 1 = benigno

print("Forma de X (filas, columnas):", X.shape)
print("Clases:", (i: str(n) for i, n in enumerate(datos.target_names)))
print("Cantidad por clase:\n", y.value_counts().rename({0: "maligno (0)", 1: "benigno (1)"}))
X.head()

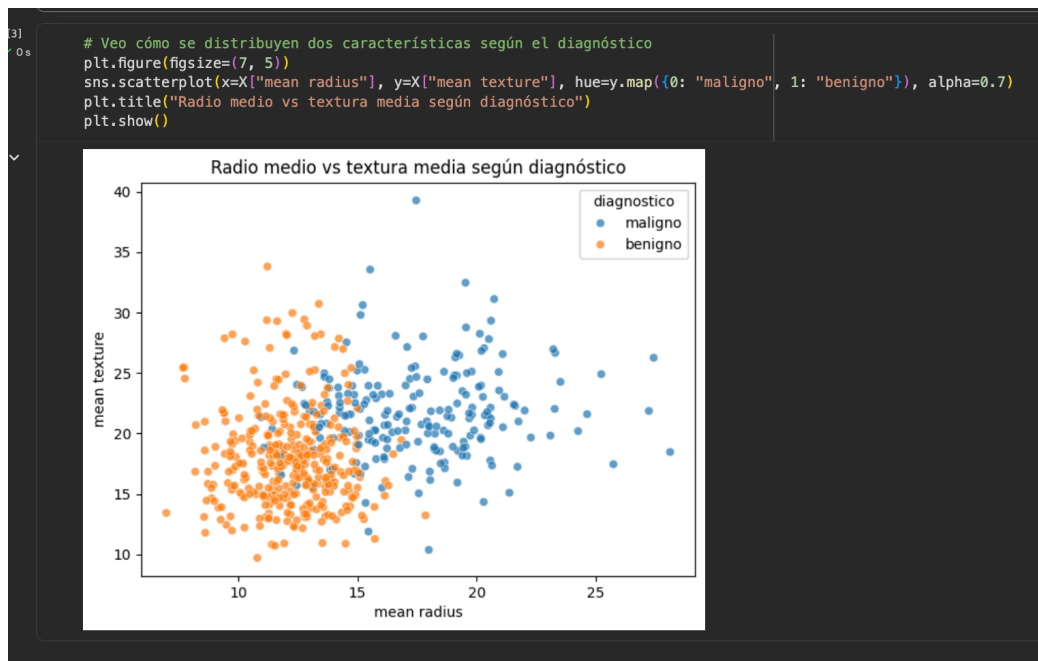
... Forma de X (filas, columnas): (569, 30)
Clases: {0: 'malignant', 1: 'benign'}
Cantidad por clase:
diagnostico
benigno (1)    357
maligno (0)    212
Names: count, dtype: int64

   mean radius    mean texture    mean perimeter    mean area    mean smoothness    mean compactness    mean concavity    mean concave points    mean symmetry    mean fractal dimension    ...    worst radius    worst texture    worst perimeter    worst area    worst smoothness    worst compactness    worst concavity    worst concave points    worst symmetry    worst fractal dimension
0      17.99         10.38         122.80        1001.0         0.11840         0.27780         0.3001         0.14710         0.2419         0.07871    ...         25.38         17.33         184.80        2019.0         0.1622         0.8668         0.7119         0.2854         0.4801         0.11890
1      20.57         17.77         132.90        1326.0         0.08474         0.07864         0.0869         0.07017         0.1812         0.05067    ...         24.99         23.41         158.80        1956.0         0.1238         0.1866         0.2416         0.1860         0.2750         0.08902
2      19.69         21.25         130.00        1203.0         0.10960         0.15990         0.1974         0.12790         0.2069         0.05999    ...         23.57         25.53         152.50        1709.0         0.1444         0.4245         0.4504         0.2430         0.3613         0.08758
3      11.42         20.38          77.58         386.1         0.14250         0.28390         0.2414         0.10520         0.2597         0.09744    ...         14.91         26.50         98.67         567.7         0.2098         0.8663         0.6869         0.2575         0.6638         0.17300
4      20.29         14.34         135.10        1297.0         0.10030         0.13280         0.1980         0.10430         0.1809         0.05883    ...         22.54         16.67         152.20        1575.0         0.1374         0.2050         0.4000         0.1625         0.2364         0.07678

5 rows x 30 columns
```

Captura 2. Carga del dataset: forma, clases y primeras filas.

Para hacerme una idea de los datos dibujé el radio medio contra la textura media separando por diagnóstico. Se ve que los tumores malignos tienden a tener radio mayor, aunque las clases se traslapan: por eso hace falta un modelo con las 30 variables y no una regla simple.



Captura 3. Radio medio vs textura media por diagnóstico.

3.2 División y escalado

Separé entrenamiento y prueba (70/30, con semilla 42 para que el resultado sea reproducible) y escalé las características con StandardScaler. El escalador se ajusta solo con el entrenamiento y luego se aplica a la prueba, para no “filtrar” información del conjunto de prueba. SVM, KNN y la regresión logística lo necesitan; los árboles y Naive Bayes no.

```

2. Dividir en entrenamiento y prueba y escalar

Entreno con el 70 % de los datos y guardo el 30 % para evaluar con datos que el modelo nunca vio. Escalo las características porque
SVM, KNN y la regresión logística son sensibles a la escala.

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42, stratify=y)

escalador = StandardScaler()
X_train_esc = escalador.fit_transform(X_train) # aprende la escala con entrenamiento
X_test_esc = escalador.transform(X_test) # y la aplica a prueba

print("Entrenamiento:", X_train.shape[0], "pacientes")
print("Prueba:", X_test.shape[0], "pacientes")

Entrenamiento: 398 pacientes
Prueba: 171 pacientes

```

Captura 4. División 70/30 y escalado.

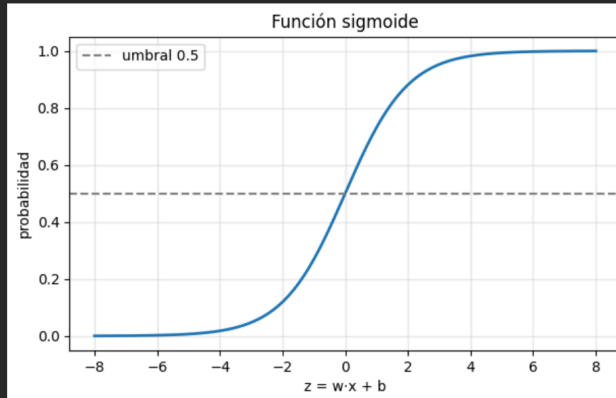
3.3 Regresión Logística

Primero dibujé la función sigmoide para entender qué hace el modelo, y luego hice un ejercicio con una sola variable (el radio medio) para poder graficar la curva de probabilidad sobre los datos. El coeficiente aprendido fue $w = -1,027$ y el intercepto $b = 15,116$: el signo negativo significa que a mayor radio, menor probabilidad de que sea benigno. Con una sola variable la exactitud fue de 0,86.

3. Regresión Logística

Aunque se llama *regresión*, sirve para clasificar: calcula la probabilidad de que un caso sea de la clase 1 usando la función sigmoide y decide con un umbral de 0,5.

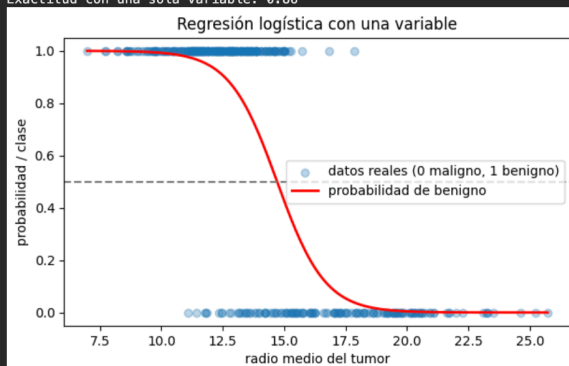
```
[5]  
✓ 0s # 3.1 La función sigmoide: convierte cualquier número en una probabilidad entre 0 y 1  
z = np.linspace(-8, 8, 200)  
sigmoide = 1 / (1 + np.exp(-z))  
  
plt.figure(figsize=(7, 4))  
plt.plot(z, sigmoide, linewidth=2)  
plt.axhline(0.5, color="gray", linestyle="--", label="umbral 0.5")  
plt.title("Función sigmoide")  
plt.xlabel("z = w·x + b"); plt.ylabel("probabilidad")  
plt.legend(); plt.grid(alpha=0.3)  
plt.show()
```



Captura 5. Función sigmoide.

```
# 3.2 Ejercicio sencillo: regresión logística con UNA sola característica (radio medio)  
x1_train = X_train[["mean radius"]].values  
x1_test = X_test[["mean radius"]].values  
  
log_simple = LogisticRegression()  
log_simple.fit(x1_train, y_train)  
  
print("Coeficiente w:", log_simple.coef_[0][0].round(3))  
print("Intercepto b:", log_simple.intercept_[0].round(3))  
print("Exactitud con una sola variable:", round(accuracy_score(y_test, log_simple.predict(x1_test)), 3))  
  
# Dibujo la curva de probabilidad sobre los datos  
rango = np.linspace(x1_train.min(), x1_train.max(), 200).reshape(-1, 1)  
plt.figure(figsize=(7, 4))  
plt.scatter(x1_train, y_train, alpha=0.3, label="datos reales (0 maligno, 1 benigno)")  
plt.plot(rango, log_simple.predict_proba(rango)[:, 1], color="red", linewidth=2, label="probabilidad de benigno")  
plt.axhline(0.5, color="gray", linestyle="--")  
plt.xlabel("radio medio del tumor"); plt.ylabel("probabilidad / clase")  
plt.title("Regresión logística con una variable")  
plt.legend(); plt.show()
```

Coeficiente w: -1.027
Intercepto b: 15.116
Exactitud con una sola variable: 0.86



Captura 6. Regresión logística con una sola variable: coeficientes y curva de probabilidad.

Con las 30 características el modelo mejoró muchísimo: **exactitud 0,9883, precisión 0,9907, recall 0,9907 y F1 0,9907**. La matriz de confusión muestra solo dos errores en 171 pacientes: 1 falso positivo y 1 falso negativo.



Captura 7. Regresión logística con 30 características: métricas, reporte y matriz de confusión.

3.4 Función de evaluación

Para no repetir el mismo código seis veces, escribí una función evaluar() que entrena el modelo, calcula las cuatro métricas, imprime el reporte de clasificación, dibuja la matriz de confusión y guarda los resultados en un diccionario para la comparación final.

```

4. Modelos de clasificación

Para no repetir código, defino una función que entrena un modelo, calcula las métricas y dibuja su matriz de confusión.

resultados = {} # aquí guardo las métricas de cada modelo para compararlos al final

def evaluar(nombre, modelo, Xtr, Xte):
    modelo.fit(Xtr, y_train)
    pred = modelo.predict(Xte)
    métricas = {
        "Exactitud": accuracy_score(y_test, pred),
        "Precisión": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1": f1_score(y_test, pred),
    }
    resultados[nombre] = métricas
    print(f"=== {nombre} ===")
    for k, v in métricas.items():
        print(f"{k}: {v:.4f}")
    print()
    print(classification_report(y_test, pred, target_names=["maligno", "benigno"]))
    ConfusionMatrixDisplay(confusion_matrix(y_test, pred), display_labels=["maligno", "benigno"]).plot(cmap="Blues")
    plt.title(f"Matriz de confusión - {nombre}")
    plt.show()
    return modelo

# Guardo también la regresión logística en la tabla de resultados
resultados["Regresión logística"] = {
    "Exactitud": accuracy_score(y_test, pred_log), "Precisión": precision_score(y_test, pred_log),
    "Recall": recall_score(y_test, pred_log), "F1": f1_score(y_test, pred_log)}
print("Función lista.")

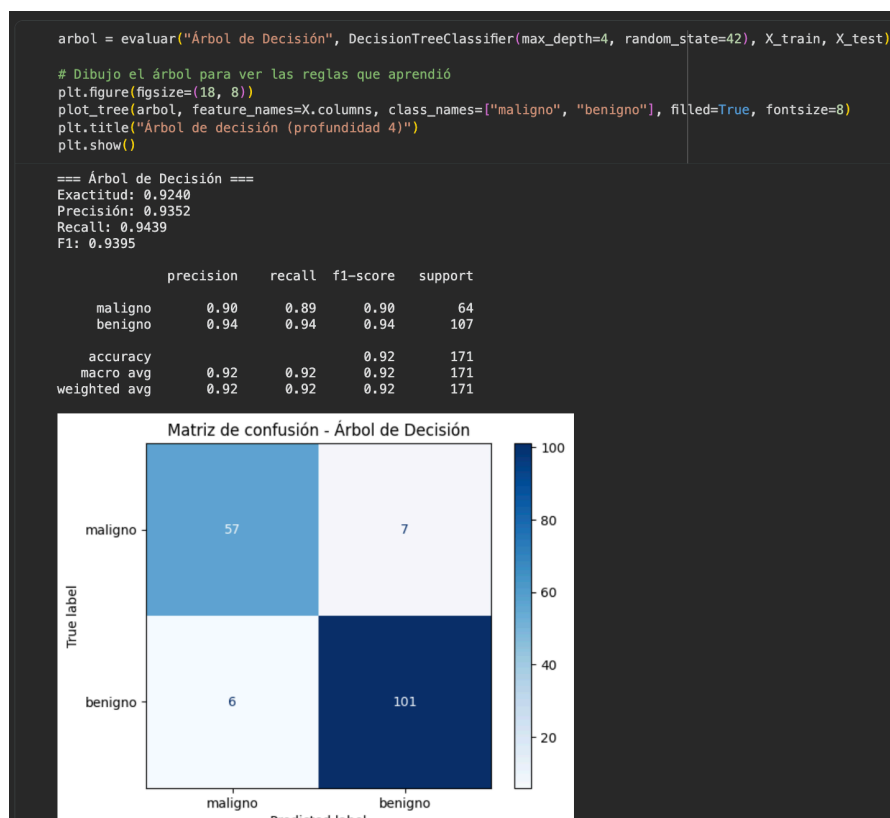
... Función lista.

```

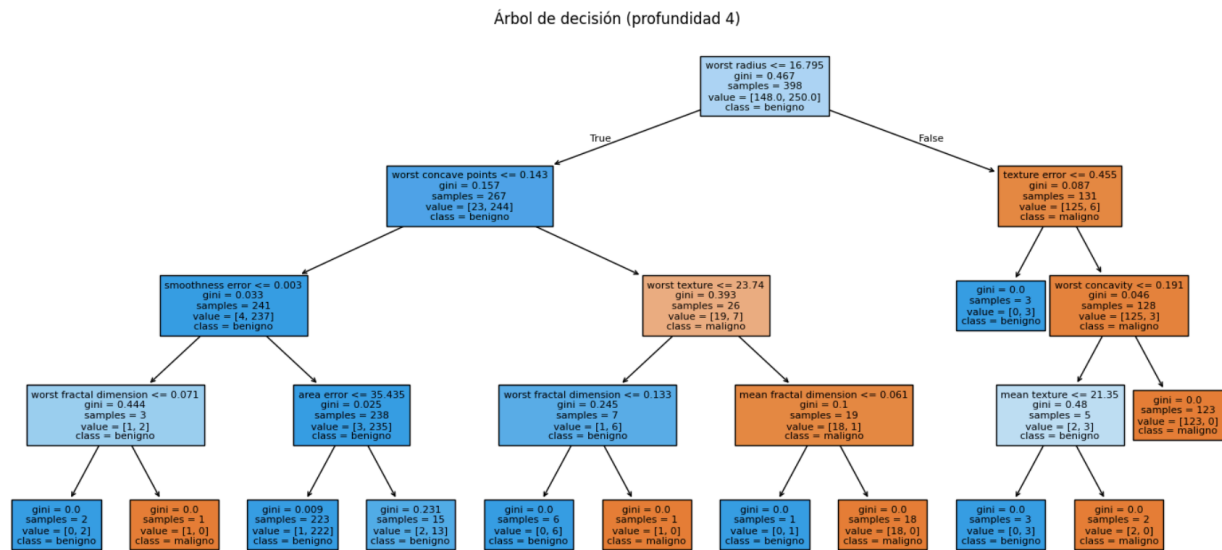
Captura 8. Función de evaluación reutilizable.

3.5 Árbol de Decisión

Usé un árbol de profundidad 4 para que se pueda leer. Obtuvo exactitud 0,9240 y F1 0,9395, con 7 falsos positivos y 6 falsos negativos. Lo valioso es que se puede dibujar: la primera pregunta que hace es si el “worst concave points” (peor valor de puntos cóncavos) es menor o igual a cierto umbral, y a partir de ahí va preguntando por el área y el radio. Es el modelo que un médico podría leer sin saber programar.



Captura 9. Árbol de decisión: métricas, matriz de confusión y reglas aprendidas.



Captura 10. Árbol de decisión

3.6 Random Forest

Con 200 árboles el bosque subió a exactitud 0,9415 y F1 0,9537, con 6 falsos positivos y 4 falsos negativos. Además entrega la importancia de cada característica: las más influyentes fueron las medidas “worst” (peores valores) de puntos cóncavos, área, radio y perímetro, lo que coincide con el árbol individual.

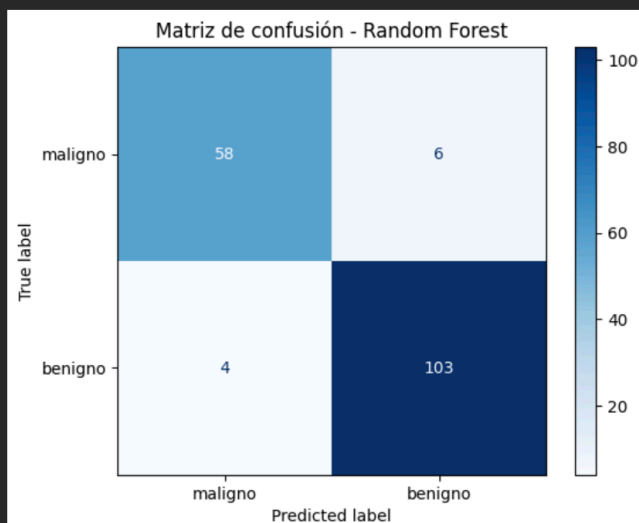
4.2 Random Forest

Combina muchos árboles entrenados con muestras distintas y vota. Suele ser más preciso y estable que un solo árbol.

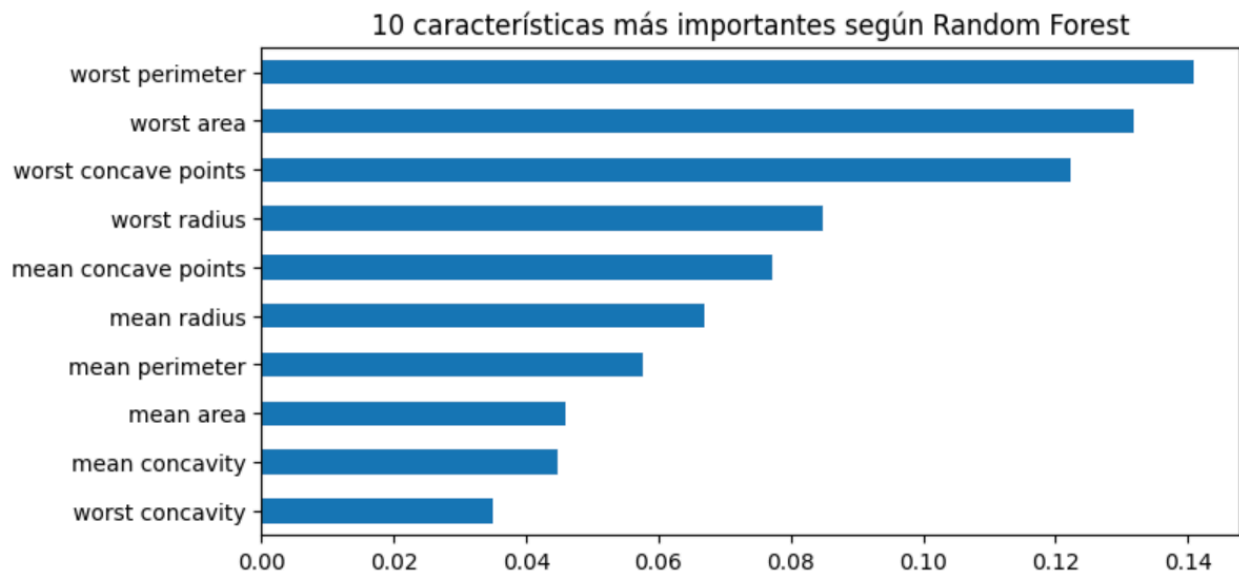
```
bosque = evaluar("Random Forest", RandomForestClassifier(n_estimators=200, random_state=42), X_train, X_test)

# ¿Qué características importan más para el bosque?
importancias = pd.Series(bosque.feature_importances_, index=X.columns).sort_values(ascending=False).head(10)
plt.figure(figsize=(8, 4))
importancias.sort_values().plot(kind="barh")
plt.title("10 características más importantes según Random Forest")
plt.show()
```

	precision	recall	f1-score	support
maligno	0.94	0.91	0.92	64
benigno	0.94	0.96	0.95	107
accuracy			0.94	171
macro avg	0.94	0.93	0.94	171
weighted avg	0.94	0.94	0.94	171



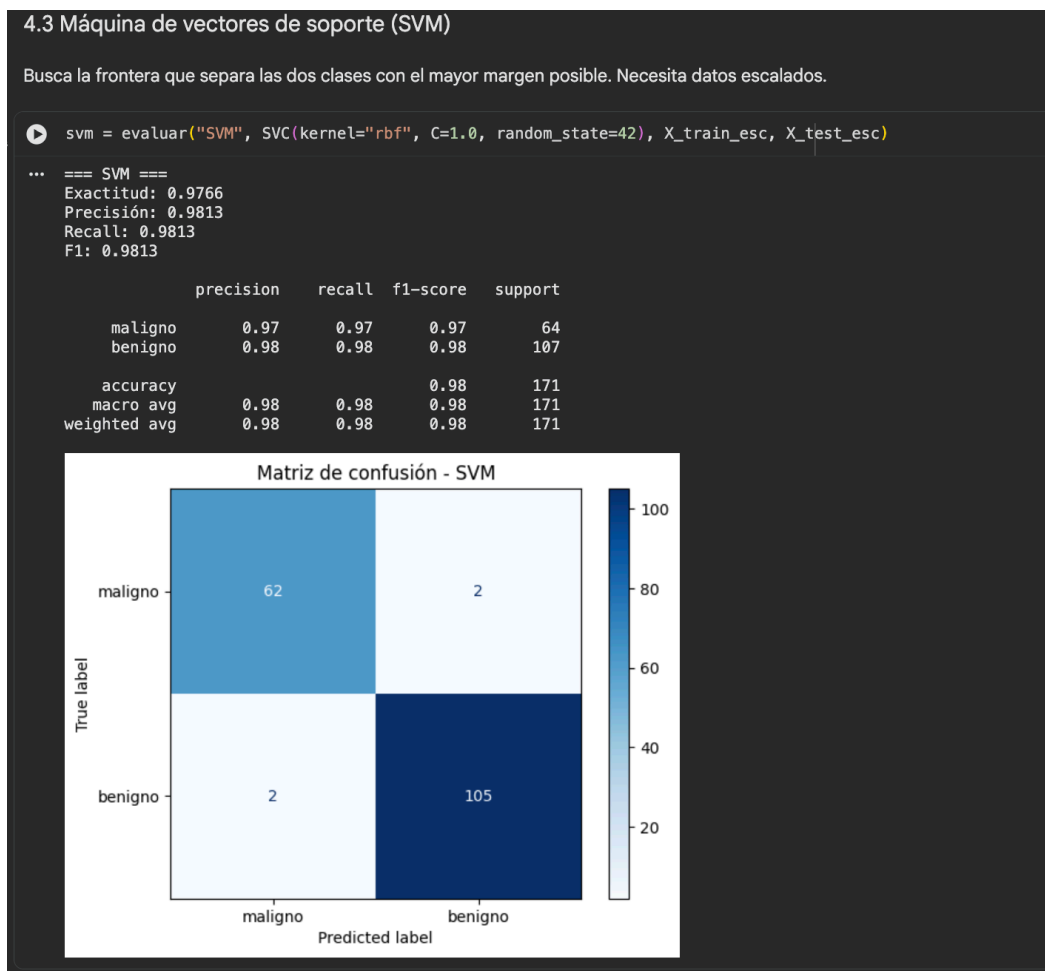
Captura 11. Random Forest: métricas, matriz de confusión



Captura 12. Características mas importantes según Random Forest

3.7 Máquina de vectores de soporte (SVM)

Con kernel RBF y $C = 1$ sobre los datos escalados, la SVM obtuvo exactitud 0,9766 y F1 0,9813: 2 falsos positivos y 2 falsos negativos. Es el segundo mejor modelo y muy equilibrado entre las dos clases.



Captura 13. SVM: métricas y matriz de confusión.

3.8 K vecinos más cercanos (KNN)

Probé k de 1 a 20 y el mejor fue $k = 12$. Con ese valor KNN alcanzó exactitud 0,9766, precisión 0,9640, recall 1,0000 y F1 0,9817. Su matriz es especial: no tuvo ningún falso negativo (encontró los 107 benignos), pero clasificó 4 tumores malignos como benignos. Esto muestra que una exactitud igual a la de la SVM puede esconder errores de tipo distinto.

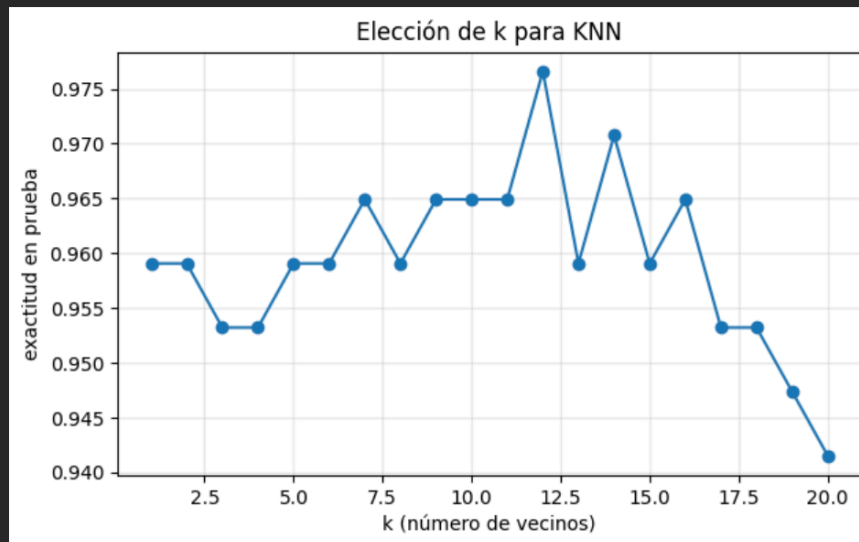
```

# Pruebo distintos valores de k
exactitudes = []
for k in range(1, 21):
    knn_k = KNeighborsClassifier(n_neighbors=k).fit(X_train_esc, y_train)
    exactitudes.append(accuracy_score(y_test, knn_k.predict(X_test_esc)))

mejor_k = int(np.argmax(exactitudes)) + 1
plt.figure(figsize=(7, 4))
plt.plot(range(1, 21), exactitudes, marker="o")
plt.xlabel("k (número de vecinos)"); plt.ylabel("exactitud en prueba")
plt.title("Elección de k para KNN"); plt.grid(alpha=0.3)
plt.show()
print("Mejor k:", mejor_k)

knn = evaluar("KNN", KNeighborsClassifier(n_neighbors=mejor_k), X_train_esc, X_test_esc)

```



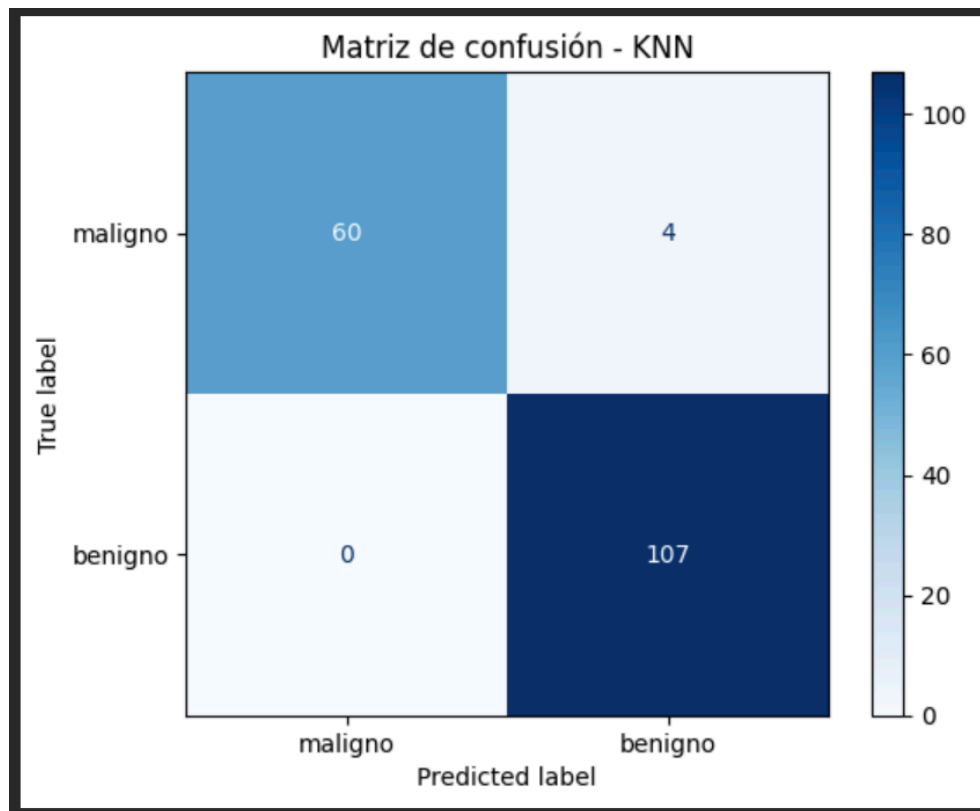
```

Mejor k: 12
=== KNN ===
Exactitud: 0.9766
Precisión: 0.9640
Recall: 1.0000
F1: 0.9817

```

	precision	recall	f1-score	support
maligno	1.00	0.94	0.97	64
benigno	0.96	1.00	0.98	107
accuracy			0.98	171
macro avg	0.98	0.97	0.97	171
weighted avg	0.98	0.98	0.98	171

Captura 14. KNN: elección de k , métricas y matriz de confusión.



Captura 14. KNN matriz de confusión.

3.9 Naive Bayes

GaussianNB, sin ningún parámetro, obtuvo exactitud 0,9474 y F1 0,9589, con 7 falsos positivos y 2 falsos negativos. Es el modelo más rápido de entrenar y, aunque supone que las 30 características son independientes (cosa que no es cierta: radio, perímetro y área están muy relacionados), funciona razonablemente bien.

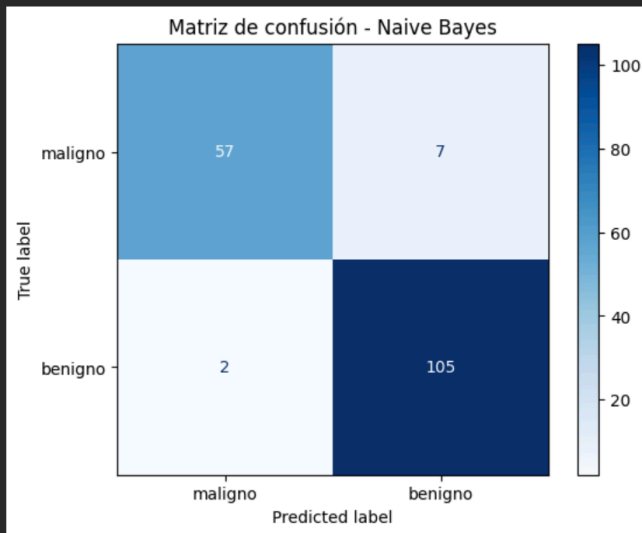
4.5 Naïve Bayes

Usa el teorema de Bayes suponiendo que las características son independientes entre sí. Es muy rápido y no tiene casi parámetros.

```
nb = evaluar("Naive Bayes", GaussianNB(), X_train, X_test)
```

```
... === Naive Bayes ===  
Exactitud: 0.9474  
Precisión: 0.9375  
Recall: 0.9813  
F1: 0.9589
```

	precision	recall	f1-score	support
maligno	0.97	0.89	0.93	64
benigno	0.94	0.98	0.96	107
accuracy			0.95	171
macro avg	0.95	0.94	0.94	171
weighted avg	0.95	0.95	0.95	171



Captura 15. Naive Bayes: métricas y matriz de confusión.

4. Análisis de resultados

La tabla y la gráfica final del cuaderno comparan los seis modelos sobre el mismo conjunto de prueba de 171 pacientes.

5. Comparación de todos los modelos

```

tabla = pd.DataFrame(resultados).T.round(4)
tabla = tabla.sort_values("F1", ascending=False)
print(tabla)

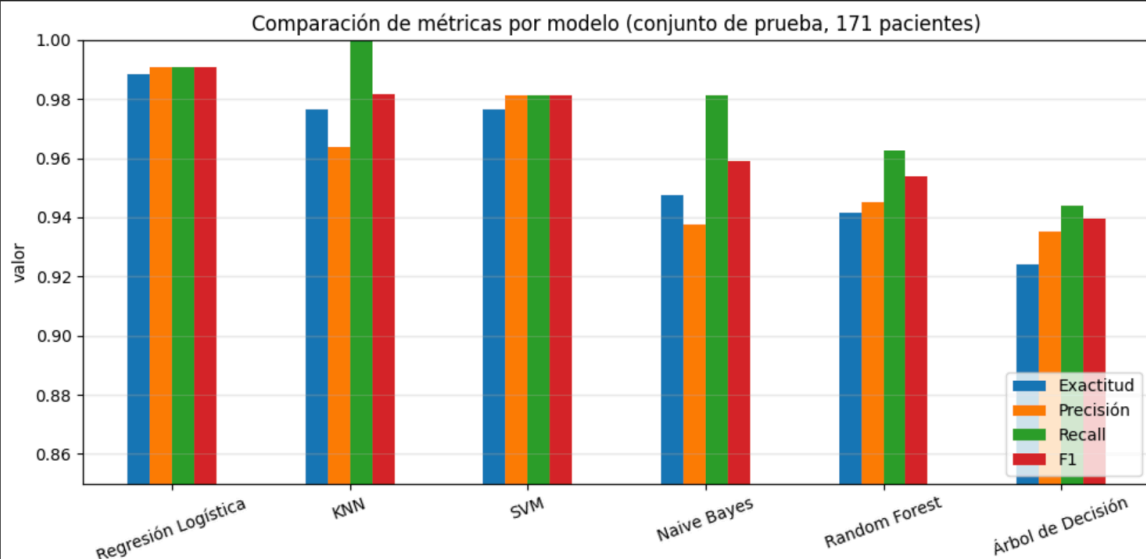
tabla.plot(kind="bar", figsize=(10, 5), ylim=(0.85, 1.0))
plt.title("Comparación de métricas por modelo (conjunto de prueba, 171 pacientes)")
plt.ylabel("valor"); plt.xticks(rotation=20); plt.legend(loc="lower right"); plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

```

```

...
Exactitud  Precisión  Recall    F1
Regresión Logística  0.9883    0.9907  0.9907  0.9907
KNN            0.9766    0.9640  1.0000  0.9817
SVM            0.9766    0.9813  0.9813  0.9813
Naive Bayes    0.9474    0.9375  0.9813  0.9589
Random Forest  0.9415    0.9450  0.9626  0.9537
Árbol de Decisión  0.9240    0.9352  0.9439  0.9395

```



Captura 16. Comparación de métricas de los seis modelos.

Modelo	Exactitud	Precisión	Recall	F1	Errores (FP / FN)
Regresión Logística	0,9883	0,9907	0,9907	0,9907	1 / 1
SVM (RBF)	0,9766	0,9813	0,9813	0,9813	2 / 2
KNN (k = 12)	0,9766	0,9640	1,0000	0,9817	4 / 0
Naive Bayes	0,9474	0,9375	0,9813	0,9589	7 / 2
Random Forest (200)	0,9415	0,9450	0,9626	0,9537	6 / 4
Árbol de Decisión (prof. 4)	0,9240	0,9352	0,9439	0,9395	7 / 6

Tabla 1. Métricas sobre el conjunto de prueba. FP = malignos clasificados como benignos; FN = benignos clasificados como malignos (clase positiva = benigno).

¿Cuál fue el mejor modelo? La regresión logística, con solo 2 errores en 171 pacientes. Me sorprendió que el modelo más simple le ganara a Random Forest, pero tiene explicación: las 30 características de este dataset separan las clases de forma casi lineal una vez escaladas, y en ese escenario un modelo lineal bien regularizado es difícil de superar (Géron, 2019). SVM quedó muy cerca.

¿Qué métrica importa más aquí? En este problema el error grave es el falso positivo tal como lo definí (decir “benigno” cuando el tumor es maligno), porque deja a una paciente sin tratamiento. Mirando esa columna, la regresión logística (1) y la SVM (2) son las más seguras, mientras que Naive Bayes y el árbol

se equivocan 7 veces. KNN tiene recall perfecto para benignos, pero 4 malignos se le escaparon; si yo fuera el médico, preferiría la SVM aunque tenga un F1 ligeramente menor.

¿Por qué Random Forest no superó al árbol por más? Porque el árbol ya estaba limitado a profundidad 4 y el bosque usa parámetros por defecto; con más ajuste de hiperparámetros seguramente mejoraría. Aun así el bosque fue más estable y, de regalo, entregó la importancia de las variables, que coincide con la intuición médica: los peores valores de puntos cóncavos, área y radio son los que más pesan.

Interpretabilidad vs precisión. El árbol es el único modelo que se puede explicar con una imagen; la regresión logística también es interpretable (cada coeficiente dice cuánto aporta cada variable), pero SVM y KNN son cajas negras. Si el objetivo fuera apoyar a un médico, combinaría la precisión de la regresión logística con las reglas del árbol para explicar los casos.

5. Conclusiones

- Implementé los seis algoritmos de la guía sobre el mismo dataset y los comparé con las mismas métricas. Todos superaron el 92 % de exactitud, lo que indica que el problema está bien planteado y los datos bien etiquetados.
- La regresión logística fue el mejor modelo (F1 0,9907) y la SVM el segundo (F1 0,9813). El modelo más simple ganó, lo que confirma que no siempre el algoritmo más complejo es la mejor opción: depende de los datos.
- La exactitud sola engaña. KNN y SVM tienen la misma exactitud (0,9766) pero cometen errores de tipo distinto; solo la matriz de confusión muestra cuál es más seguro para el caso médico.
- Escalar los datos, dividir con estratificación y fijar la semilla aleatoria fueron detalles pequeños que hicieron que los resultados fueran comparables y reproducibles.
- Como siguientes pasos probaría validación cruzada en lugar de una sola división 70/30, ajustaría los hiperparámetros de Random Forest y SVM con GridSearchCV, y mediría el tiempo de entrenamiento de cada modelo como criterio adicional.

Referencias

- Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (2.^a ed.). O'Reilly Media.
- Gupta, V., Mishra, V. K., Singhal, P., & Kumar, A. (2022). An overview of supervised machine learning algorithm. En *Proceedings of the 11th International Conference on System Modeling & Advancement in Research Trends (SMART 2022)* (pp. 87–92). IEEE.
<https://ieeexplore.ieee.org/document/10047618>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Scikit-learn developers. (s. f.). *Breast cancer Wisconsin (diagnostic) dataset*. https://scikit-learn.org/stable/datasets/toy_dataset.html#breast-cancer-dataset

Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). *Breast cancer Wisconsin (diagnostic) data set* [Dataset]. UCI Machine Learning Repository.
<https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>